

游龙排排 · 智能任务规划器

项目计划书与 AI 工具使用声明（合并版）

课程：开源商业分析

项目类型：基于大模型 Agent 的智能任务规划 Web 应用

技术栈：Next.js 16 · React 19 · TypeScript · Tailwind CSS v4 · SQLite

核心能力：目标拆解 · 联网取证 · 结构化生成 · 看板/时间轴 · 多格式导出

文档构成：第一至十章为项目报告；第十一章为 AI 工具使用声明

目录

本文档是「项目计划书」与「AI 工具使用声明」的合并版本。第一至第十章为项目报告正文，系统阐述项目背景、需求、技术选型、架构与数据模型、Agent 实现细节、功能与界面展示、工程质量、问题与解决、部署方式以及总结展望；第十一章为独立的 AI 工具使用声明，专门说明在开发过程中如何使用 AI 辅助工具（而非项目本身的 AI 功能）。

- ☐ 第一章 项目背景与目标
- ☐ 第二章 需求分析与功能范围
- ☐ 第三章 技术选型与理由
- ☐ 第四章 系统架构与数据模型（含 ER 图）
- ☐ 第五章 Agent 规划链路的实现细节
- ☐ 第六章 关键功能与界面展示（含运行截图）
- ☐ 第七章 工程质量：测试、构建与提交规范
- ☐ 第八章 开发过程中的问题与解决方案
- ☐ 第九章 部署与一键启动
- ☐ 第十章 总结与展望
- ☐ 第十一章 AI 工具使用声明（开发过程）
- ☐ 附录 验证命令与目录结构

第一章 项目背景与目标

1.1 选题来由

在学习、工作与日常生活中，人们经常面对一类「目标清晰但路径模糊」的复杂任务：比如「用三千元预算策划一次三天的北京周末游，偏好文化景点和本地美食」，或者「一个月内准备大学英语四级，每天学习时间不超过两小时」。这类目标的共同特征是：一句话就能说清楚「想要什么」，但要真正落地，需要把它拆成一连串可执行、有先后顺序、彼此存在依赖关系的具体步骤，并且往往还需要查询一些外部公开信息（票务、攻略、时间安排、注意事项）才能把计划做得靠谱。

传统的待办清单工具只解决「记录」，不解决「拆解」；通用聊天机器人能给出一段文字建议，但输出是非结构化的，难以编辑、排序、追踪进度，也缺乏对信息来源可信度的标注与对个人偏好的沉淀。游龙排排正是针对这一空白而设计：它把大语言模型的「理解与拆解能力」、本地搜索引擎的「取证能力」与结构化数据库的「持久化与管理能力」组合起来，形成一个可以反复使用、越用越懂用户的「智能任务规划器」。

1.2 项目目标

项目的总体目标是：用户输入一个复杂目标后，系统通过一条轻量的 Agent 状态机自动完成意图识别、槽位抽取、补充问题生成、（可选的）联网取证与结构化计划生成，最终产出一份可编辑、可拖拽排序、可标注依赖与截止时间、可多格式导出的待办清单，并把每一次生成的过程、来源与运行日志完整保存，供后续查看、复盘与记忆复用。

- 可用性目标：在没有任何外部密钥、没有 Docker 的「裸机」环境下也能跑通核心流程（自动降级为本地启发式拆解），保证离线评审与本地演示稳定。
- 可解释性目标：每一步生成都附带阶段日志、假设与追问、来源证据及其可信度分级，让结果「可被理解、可被质疑、可被复核」。
- 工程化目标：具备类型检查、单元测试、构建校验与提交包检查，能够以「一条命令」完成全量验收。
- 体验目标：提供面向「Agent 工具」的指挥中心式界面，支持白天/夜间双主题与 Cmd+K 命令面板等专业交互。

第二章 需求分析与功能范围

2.1 目标用户与典型场景

目标用户是需要规划复杂个人事务的学生与职场人士。系统内置了七类典型场景模板：周末旅行、考试备考、班级团建、求职准备、采购清单、健康习惯与课程项目。用户既可以从模板一键填充目标，也可以完全自由输入。每一类目标在「补充问题生成」与「搜索查询构造」上会有差异化处理，使拆解结果更贴合该领域的常识。

2.2 功能性需求

围绕「输入目标 → 生成计划 → 管理执行 → 导出复盘」这条主线，系统实现了以下核心功能：

- 目标输入与拆解：接收自然语言目标与可选条件（截止时间、预算、地点、偏好、限制），输出带子任务树的结构化计划。
- 模板库：七类内置场景（旅行、备考、团建、求职、采购、健康、课程项目）一键填充。
- Agent 记忆：按目标类型沉淀偏好与约束，并在同类目标生成时检索注入。
- 流式阶段反馈：通过 SSE 实时展示意图识别、取证、生成等阶段日志。
- 联网搜索与可信度分级：通过本地 SearXNG 取回来源，并按域名与关键词给出高/中/低/待复核分级与引用理由。
- 任务看板：子任务树、拖拽排序、状态切换、优先级标签。
- 依赖与时间轴：主任务按截止时间排序，依赖未满足时标记「待前置」。
- 历史管理：搜索、类型筛选、归档、复制、删除确认。
- 导出中心：Markdown、Word、PDF、Excel 与提交素材包 ZIP，导出前自动扫描敏感信息。
- 报告助手与复盘面板：生成报告草稿，记录完成率与延期原因并反哺记忆。
- 设置与健康检查：可写应用偏好、工具健康检查、一键清库；可选访问密码保护。

2.3 非功能性需求

- 鲁棒性：模型或搜索任一环节失败都不应中断整体流程，须自动降级并给出明确提示。
- 隐私：API Key 仅在服务端读取，绝不进入前端 bundle；用户数据只落本地 SQLite；敏感输入在导出与提交前给出脱敏提示。
- 确定性：界面渲染（尤其是时间）必须在服务端与客户端一致，避免 hydration 抖动。
- 可移植：依赖通过 package.json 精确描述，配合一键脚本可在裸机还原环境。

2.4 一个典型用户故事

以「策划三天北京周末游、预算三千元、偏好文化景点和本地美食」为例：用户在首页点选「周末旅行」模板自动填入目标，勾选联网搜索后点击生成。系统先识别出这是一个旅行类目标，抽取出时间（三天）、预算（三千元）、地点（北京）与偏好（文化、美食）等槽位，发现缺少「出发地、住宿区域、饮食忌口」等信息便生成补充问题；随后构造若干搜索查询，通过本地 SearXNG 取回攻略类来源并做可信度分级；接着把目标、槽位、记忆与来源摘要一起交给 DeepSeek，得到一份含「按天行程、住宿预订、预算分配」的结构化计划。

用户在计划详情页可以展开每一天的子任务、拖拽调整顺序、勾选完成状态、查看每条来源的可信度，最后一键导出为 Word 或 PDF 带去打印，或导出「提交素材包 ZIP」。执行结束后还能在复盘面板记录「哪几项延期、原因是什么」，这些结论会沉淀进记忆中心——下次再规划同类旅行时，系统会自动参考这些偏好。这条从「一句话目标」到「可执行、可追踪、可复盘」的完整闭环，正是项目要解决的核心问题。

第三章 技术选型与理由

3.1 总体技术栈

项目是一个纯 Node.js 应用，不含任何 Python 代码，运行时只依赖 Node.js。前后端统一在 Next.js (App Router) 中实现，前端为 React 19 + TypeScript + Tailwind CSS v4，后端为 Next.js 的服务端组件与 Route Handlers，数据层为 SQLite (通过 Drizzle ORM 与 better-sqlite3 同步驱动)，大模型为 DeepSeek 的 Chat Completion API，搜索为本地自建的 SearXNG。

3.2 选型理由

- Next.js App Router: 一个仓库同时承载页面与 API，服务端组件天然适合「读数据库 + 直出 HTML」，减少前后端样板与跨服务部署成本，契合课程项目的体量。
- TypeScript + Zod: 在编译期与运行期双重把关。Zod 用于校验模型返回的 JSON 结构，确保「不可信的模型输出」在落库前被结构化约束。
- SQLite + Drizzle + better-sqlite3: 单文件、零运维、同步 API，非常适合本地优先的个人工具；Drizzle 提供类型安全的表定义与查询。
- DeepSeek API: 以较低成本提供稳定的中文结构化生成能力，并支持 JSON 输出，便于约束格式。
- 本地 SearXNG: 自托管的元搜索引擎，避免把用户目标直接送往商业搜索 API，兼顾隐私与可控性；通过 Docker Compose 一键拉起。
- Tailwind CSS v4: 以 CSS 变量为核心的主题系统，便于实现「一套工具类、双主题整体翻转」。
- 导出生态: docx、pdf-lib + fontkit (中文 PDF)、ExcelJS、JSZip，均为成熟的纯 JS 库，无需系统级依赖。

3.3 备选方案与取舍

在数据层，曾考虑过用 Prisma 搭配 SQLite，但 Prisma 在本地需要额外的查询引擎二进制并以异步为主，对这种轻量本地工具略显笨重；最终选择 Drizzle + better-sqlite3 的同步组合，类型推导直接、零额外进程，配合一个原生模块自检脚本即可规避 Node 版本升级带来的 ABI 不匹配。

在搜索层，直接调用第三方搜索 API 虽然省事，但会把用户的原始目标（可能包含隐私）送往外部服务，且受配额与计费约束；自托管 SearXNG 把这部分能力本地化，既保护隐私又便于在离线环境下降级运行。在模型层，之所以选择 DeepSeek 而非自部署开源模型，是因为后者对显卡与运维要求高，不适合课程项目的演示门槛；同时我们把模型层的调用收敛在 deepseek.ts 一个文件中，未来若要替换为其他兼容 OpenAI 协议的模型，改动面很小。

第四章 系统架构与数据模型

4.1 分层架构

系统在逻辑上分为四层：表现层（src/app 页面与 src/components 组件）、接口层（src/app/api 下的 Route Handlers）、领域层（src/lib/agent 编排、src/lib/search 搜索与可信度、src/lib/export 导出、src/lib/templates 模板、src/lib/reports 报告）与数据层（src/lib/db 的 schema、queries 与初始化）。表现层只负责渲染与交互，所有可信逻辑（模型调用、校验、落库）都在服务端完成。

4.2 数据模型与 ER 图

数据层以「计划（plans）」为聚合根，围绕它组织十张表。任务表（tasks）通过自引用 parent_task_id 形成子任务树，任务之间的依赖以 JSON 数组形式保存依赖任务的 id；证据（evidence_items）、运行日志（agent_runs）、复盘（plan_reviews）、导出任务（export_jobs）都以外键挂在计划之下；用户记忆（user_memory）、模板（templates）、搜索缓存（search_cache）与键值设置（settings）相对独立。下图为数据库实体关系图：

游龙排排 · 数据库实体关系图 (SQLite)

以「计划 plans」为聚合根，任务自引用形成子任务树；证据、运行日志、复盘、导出任务均挂在计划下。

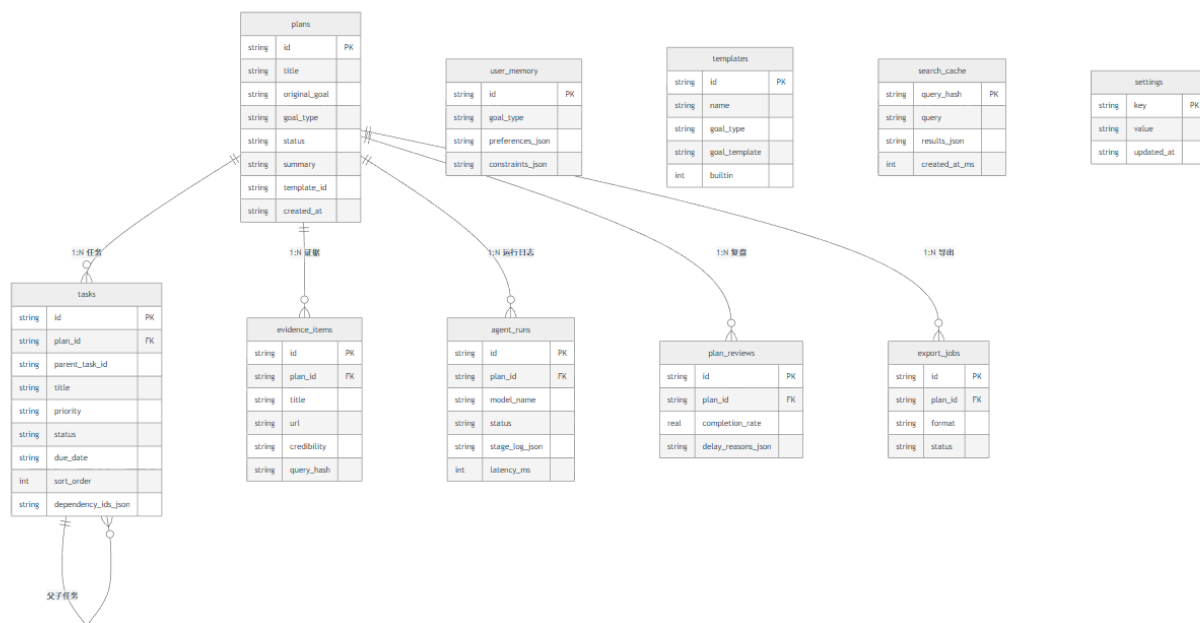


图 4-1 游龙排排数据库实体关系图 (SQLite, 10 张表)

为兼顾灵活性与简单性，复杂的嵌套结构（如假设列表、补充问题、搜索词、阶段日志、依赖 id、token 用量等）统一以「*_json」文本列保存，读写经 `src/lib/db/json.ts` 收敛序列化与反序列化逻辑；时间列使用文本型时间戳（CURRENT_TIMESTAMP），在展示层用确定性格式化函数处理，避免不同运行环境的时区与本地化差异导致服务端/客户端渲染不一致。

4.3 接口层与一次请求的生命周期

接口层由 Next.js 的 Route Handlers 承担，按资源划分：`/api/plans` 负责计划的生成与读取，其中 `/api/plans/generate` 为一次性生成、`/api/plans/generate/stream` 为流式生成；`/api/export/[planId]` 负责按格式导出；`/api/reports/generate` 负责报告草稿；`/api/settings` 负责读写偏好与清库；`/api/tools/health` 负责健康检查。所有写操作都集中在服务端，前端只通过受控的接口与之交互。

以一次「生成计划」请求为例：前端把表单数据 POST 到生成接口，服务端先用 Zod 校验输入，再调用 `runPlanPipeline` 执行整条流水线；流式接口会在每个阶段完成时通过 `Server-Sent Events` 推送一条事件，前端据此更新「实时跑道」；流水线结束后，计划、任务、证据与运行日志在同一个数据库事务中写入，接口返回新计划的 `id`，前端随即跳转到计划详情页。整个过程中，模型密钥、数据库句柄等敏感资源始终只存在于服务端。

第五章 Agent 规划链路的实现细节

5.1 线性状态机与全程降级

Agent 的核心编排集中在 `src/lib/agent/orchestrator.ts` 的 `runPlanPipeline` 函数中，被刻意设计为一条「线性状态机 + 全程降级」的流水线，依次经过以下阶段：意图识别（推断目标类型）→ 槽位抽取（时间、预算、地点、偏好等）→ 补充问题生成 → 记忆检索 → 搜索规划 → SearXNG 取证 → DeepSeek 结构化生成 → schema 校验 → 时间与依赖规范化 → 事务落库 → 记忆回写。

关键设计在于：任意一步失败都不会让整个请求崩溃。若没有配置 DeepSeek 密钥或模型调用异常，系统会捕获错误并切换到本地启发式拆解（`buildFallbackPlan`），把 `modelName` 记为 `local-fallback`；若 SearXNG 不可用，则跳过取证、继续生成基础计划，并通过告警提示用户。这种「永远有兜底」的设计，使得项目在评审机、离线环境等不可控条件下依然稳定可用。

5.2 阶段日志与流式反馈

流水线每推进一步都会通过 `pushStage` 推送一条 `StageLogEntry`（包含阶段名、状态与中文说明）。这些日志一方面随本次运行写入 `agent_runs` 表用于事后审计，另一方面通过回调实时回传：在流式接口 `/api/plans/generate/stream` 中以 SSE 持续推送给前端，让用户在等待生成时看到「正在识别意图 / 正在取证 / 正在生成」的实时进度（前端以「Live Runway」竖向轨道呈现）。

5.3 结构化生成与 JSON 健壮性

模型被要求输出严格的 JSON。考虑到大模型偶尔会返回被截断或包含多余文本的结果，`src/lib/agent/parse-model-json.ts` 实现了带「修复」能力的解析：先尝试直接解析，失败时进行括号配平、去除围栏与多余前后缀等修复后再解析；解析结果统一交给 `Zod schema` 校验，只有通过校验的结构才会进入落库流程。

5.4 时间分配与依赖重映射

若用户填写了截止时间，`normalizePlanSchedule` 会把日期按任务数量均匀分摊到各主任务的 `dueDate`，并清理早于今天的过期日期。落库时，子任务先被 `flatten` 成扁平列表，模型给出的「临时依赖 id」会被重映射为数据库中真实的 UUID（`remapDependencyIds`），从而保证时间轴与依赖关系在持久化后依然正确。

5.5 来源可信度分级

`src/lib/search/credibility.ts` 用一套轻量启发式给来源打分：政府、教育、百科类域名直接判为高可信；命中「官方」「人民政府」「教育部」「统计局」等关键词会上调；其余依据是否有摘要/正文给出中、低或「待复核」。其目的并非做权威事实核验，而是给生成结果附上可解释的引用理由，提醒用户「来源仅供辅助，执行前仍需人工复核」。

5.6 记忆中心

`src/lib/agent/memory.ts` 按「目标类型」维度沉淀用户偏好与约束：生成前检索同类目标的历史偏好拼成上下文注入 Prompt，生成后再从本次输入回写记忆，使同类目标越用越贴合个人习惯。记忆仅保留最近若干条以控制上下文规模，且只存本地、可在设置页一键清空。

5.7 Prompt 与 schema 版本管理

为了让生成行为可追溯，项目把 Prompt 模板与输出 schema 都纳入版本管理（PROMPT_VERSION 与 SCHEMA_VERSION 定义在 `src/lib/agent/prompts` 下）。每次生成都会把当时使用的版本号写入 `plans` 与 `agent_runs`，因此在计划详情页能看到「Prompt v2.0.0」这样的标注。这样一来，当 Prompt 迭代后，历史计划仍能说明自己是用哪一版规则生成的，便于对比不同版本的拆解质量，也便于排查「为什么这次拆得和上次不一样」。

第六章 关键功能与界面展示

本章结合实际运行截图展示主要功能界面。整体视觉语言为「Graphite Command Center（石墨指挥中心）」，强调控制感、信息层级与专业质感，并支持白天/夜间双主题与 Cmd+K 命令面板。

6.1 首页：目标输入与模板库（夜间主题）

首页顶部是「指挥栏」式的品牌头，集成历史计划、设置、⌘K 命令面板与主题切换入口；下方依次是联网搜索状态条、目标输入表单（支持截止时间、预算、地点、偏好、限制及搜索/质量/记忆三个开关）与七类模板库。夜间主题采用柔化石墨色（非纯黑），以 1px 发丝描边与克制的青色强调营造控制台质感。



图 6-1 首页（夜间主题）：指挥栏、搜索状态条与模板库

6.2 命令面板（Cmd+K / Ctrl+K）

按下 Cmd+K（Windows 为 Ctrl+K）或点击页头按钮即可唤起命令面板，支持模糊搜索、上下键选择、回车执行与 Esc 关闭，内置页面导航（首页/历史/设置）与主题切换命令。它通过全局键盘监听与自定义事件实现，挂载在根布局，任何页面均可唤起。



图 6-2 命令面板（Cmd+K）：分组命令、键盘导航与等宽提示

6.3 白天主题

考虑到不同使用场景与阅读习惯，系统提供干净明亮的白天主题。主题以 CSS 变量整体翻转：所有 `--color-*` 令牌默认取白天值，在 `html.dark` 选择器下被夜间值整体覆盖，因此所有组件无需逐处改写即可随主题换肤；用户的选择写入 `localStorage`，并通过首屏内联脚本提前注入，杜绝刷新时的主题闪烁。



图 6-3 首页（白天主题）：同一套组件随主题整体换肤

6.4 计划详情：任务面板与依赖时间轴

计划详情页以「Mission Board」头部呈现标题、目标类型、任务数量、Prompt 版本与创建时间。其下的「时间轴与依赖（Dependency Rail）」把主任务按截止时间排序，以竖向轨道与节点编号展示，仅当真实前置任务未完成时标记「待前置」。



图 6-4 计划详情页头部与依赖时间轴

6.5 待办清单：子任务树与拖拽

待办清单支持子任务树展开/收起、拖拽排序、状态切换与优先级标签（高/中/低）。每个任务可点击查看「拆解原因、状态与关联来源」，并能就地新增子任务与主任务。



图 6-5 待办清单：子任务树、优先级标签与拖拽排序

6.6 导出中心与复盘

导出中心提供 Markdown、Word、PDF、Excel 任务表与「提交素材包 ZIP」五种格式，并可一键「生成报告草稿」；导出前会自动扫描敏感信息。复盘面板记录完成率与延期原因，并把结论反哺到记忆中心。



图 6-6 导出中心与复盘面板

6.7 历史计划库

历史页（Artifact Library）支持按关键词与目标类型筛选、显示/隐藏归档，并对每个计划提供复制、归档与删除（带确认）操作。截图中的日期「2026-05-31」由确定性格式化函数生成，服务端与客户端完全一致。



图 6-7 历史计划库：筛选、归档与复制/删除

6.8 设置与健康检查

设置页（Control Surface）分为「可写的应用偏好」（默认是否联网、默认生成模式、保存偏好、清空计划数据）与「只读的运行配置」（DeepSeek API 是否已配置、默认模型等），并提供工具健康检查与隐私说明。运行配置只读，体现「密钥不落库、不入前端」的隐私边界。



图 6-8 设置页：可写偏好、只读运行配置与健康检查

第七章 工程质量：测试、构建与提交规范

7.1 类型检查与单元测试

项目以 TypeScript 严格类型为第一道防线，pnpm typecheck 必须零错误。核心纯函数与易错逻辑均有 Vitest 单元测试覆盖，包括：模型 JSON 解析与修复、可信度打分、启发式拆解、schema 校验、Markdown/PDF 导出以及访问鉴权等，pnpm test 一次性运行整个测试矩阵。

7.2 Lint 与构建

ESLint (eslint-config-next) 统一代码规范，构建阶段会执行更严格的 React Hooks 规则检查（例如禁止在 effect 中同步 setState）。本次开发中据此重构了主题订阅与命令面板的状态管理，使 pnpm lint 与 pnpm build 均零错误零告警。

7.3 提交包检查

scripts/check-submit-package.mjs 会在提交前检查是否误将 node_modules、.env、日志、构建缓存或明显的密钥痕迹纳入提交，配合 .gitignore 共同保证「提交包干净、可被重新安装」。

```
一键全量验收: pnpm typecheck && pnpm lint && pnpm test && pnpm check:submit && pnpm build
```

7.4 可维护性与代码组织

代码按「职责单一」的原则组织：页面只做渲染与交互，业务逻辑下沉到 src/lib 的领域模块，跨模块共享的类型集中在 src/lib/types.ts。命名遵循语义化约定，关键文件（编排器、数据模型、可信度、记忆、主题切换等）都补充了文件级中文注释，说明「为什么这样设计」而非逐行解释「做了什么」。这种组织方式让新读者可以从一个模块的头部注释快速建立整体认知，也方便后续替换某一层实现而不波及其它层。

第八章 开发过程中的问题与解决方案

8.1 better-sqlite3 与 Node 版本不匹配

当机器升级 Node 后，原生模块 better-sqlite3 会报 `NODE_MODULE_VERSION` 不匹配（例如要求 137 却编译于 127）。解决方案是新增 `scripts/ensure-native-modules.mjs`，在启动前探测并自动 rebuild；一键启动脚本会在 `pnpm install` 之后调用它，必要时回退到 `pnpm rebuild better-sqlite3`。

8.2 React Hydration 不一致

排查到的真实根因有两类：其一是客户端组件使用 `toLocaleDateString` 等本地化方法渲染时间，服务端与客户端时区不同导致文本不一致；其二是主题切换需要在客户端给 `<html>` 注入 `class`。解决方案是：引入确定性时间格式化函数（不依赖时区/locale），并在 `<html>` 上使用 `suppressHydrationWarning` 配合首屏内联脚本注入主题。经核对，自动化工具注入的 `data-cursor-ref` 属性属于工具产物，并非应用代码缺陷。

8.3 主题系统如何做到「不纯黑且可切换」

最初的暗色接近纯黑，观感偏硬。重构后采用「双主题 CSS 变量」方案：把画布抬升为柔化石墨色，并以一套语义令牌驱动；主题切换用 `useSyncExternalStore` 订阅 `document` 上的 `class` 这一外部可变状态，既符合 React 官方范式，又规避了 hydration 与「effect 内同步 `setState`」的告警。

8.4 中文 PDF 导出乱码

`pdf-lib` 默认字体不含中文字形，直接导出会出现问号。解决方案是内置 `assets/fonts/SimHei.ttf`，并通过 `@pdf-lib/fontkit` 嵌入字体子集；本报告 PDF 的生成器（`scripts/build-report-pdf.mjs`）同样复用这一字体与依赖，无需任何额外的系统级工具。

8.5 裸机零环境启动

考虑到评审环境可能完全没有 Node.js，一键脚本被增强为「可自举」：检测不到 Node 时尝试用 `winget` 自动安装 22 LTS，并通过 `corepack` 激活与 `package.json` 对齐的 `pnpm` 版本，再自动安装依赖、校验原生模块并启动开发服务，全过程为 UTF-8 中文提示。

第九章 部署与一键启动

9.1 推荐方式：一键启动

强烈建议直接使用一键启动：Windows 双击 `start-youlong-paipai.bat`（脚本顶部 `chcp 65001` 保证中文不乱码），macOS/Linux 运行 `bash start-youlong-paipai.sh`。脚本会自动准备 Node.js 与 pnpm、安装全部依赖、校验原生模块，并在就绪后自动打开浏览器；联网搜索所需的 SearXNG 会在检测到 Docker 时自动拉起，未就绪时降级且首页给出明显提示。

9.2 环境锁定（替代「虚拟环境」）

本项目无 Python，因此不需要 Python 虚拟环境；与之等价的「环境隔离」通过锁定 Node 版本实现：`package.json` 的 `engines` 限定 Node 版本范围、`packageManager` 固定 pnpm 版本，并提供 `.nvmrc`。如此即便评审机器的 Node 版本与开发机不同，也能通过一键脚本（`corepack/winget`）对齐到一致的运行时。

9.3 手动开发与密钥

手动方式为 `pnpm install` 后 `pnpm dev`。DeepSeek 密钥只从服务端环境变量读取，绝不写入项目文件，也不会进入前端 bundle；未配置密钥时应用自动使用本地基础拆解。

第十章 总结与展望

10.1 工作总结

游龙排排完整实现了「自然语言目标 → 结构化、可编辑、可导出、可复盘的待办清单」这一闭环，并在可解释性（阶段日志、来源可信度）、鲁棒性（全程降级）、隐私（本地优先、密钥不外泄）与工程质量（类型/测试/构建/提交检查）四个维度上达到了可交付标准。界面上以「石墨指挥中心」视觉语言与 ☐K 命令面板、双主题切换提供了专业且现代的交互体验。

10.2 不足与展望

- ☐ 检索增强：当前可信度为启发式，未来可引入更细粒度的去重、时效性与领域权重，并支持把高可信来源的关键结论结构化回填到任务。
- ☐ 协作与同步：现为本地单机，后续可增加云端同步与多人协作。
- ☐ 模型可插拔：将模型层抽象为统一接口，支持在 DeepSeek 之外接入更多可选模型。

第十一章 AI 工具使用声明（开发过程）

本章独立声明本项目在「开发过程」中如何使用 AI 辅助工具，重点说明 AI 帮助开发了什么、以及人工承担了哪些责任；这与项目本身面向用户的 AI 功能（调用 DeepSeek 进行任务拆解）是两回事，特此区分。

11.1 使用的工具与协作方式

开发过程中主要使用了 IDE 内置的 AI 辅助编程工具（基于大语言模型）。它在本项目里扮演的是「结对编程伙伴」的角色：参与样板代码生成、错误定位、设计讨论、文档润色与自动化脚本编写，但所有产物都经过本人逐一阅读、运行验证与修改后才纳入提交。

协作方式上，本人先用自然语言描述需求与约束（例如「在不破坏现有工具类的前提下加一套可切换的白天主题」），AI 给出方案或初版代码；随后本人在本地真实运行、用浏览器截图核对效果、阅读其改动并提出修正，必要时要求 AI 解释某段实现的原因，直到自己完全理解为止。AI 更多承担「提速」与「查漏」，而方向判断、取舍与最终落地由本人把控。

11.2 AI 具体参与开发的环节

- ☐ 脚手架与样板：在搭建 Next.js 页面、组件、API Route、Drizzle 表定义与 Zod schema 时，由 AI 生成初始样板，再由本人按项目约定调整命名、结构与边界。
- ☐ Agent 链路设计讨论：就「线性状态机 + 全程降级」的流水线划分、阶段日志结构、JSON 修复与依赖 id 重映射等方案与 AI 反复讨论，最终由本人确定并写入 orchestrator.ts 等源码。
- ☐ 疑难调试：在排查 better-sqlite3 的 NODE_MODULE_VERSION 报错、React hydration 不一致、ESLint 的「effect 内同步 setState」等问题时，AI 协助分析报错与提出修复思路，本人真实运行命令、核对日志并验证修复有效。
- ☐ 界面与交互升级：双主题 CSS 变量体系、首屏无闪烁主题脚本、⌘K 命令面板、依赖时间轴等交互的实现，由 AI 提供初版并经本人在浏览器实机截图核验、逐项打磨。
- ☐ 工程化脚本：一键启动脚本（含裸机自举 Node/pnpm）、原生模块自检脚本、提交包检查脚本，以及本报告的 PDF 生成器（复用项目内的 pdf-lib 与中文字体）均在 AI 协助下编写，并由本人测试通过。
- ☐ 文档与报告：README、审计文档与本计划书的结构与文字由 AI 协助起草与润色，事实、截图、ER 图与运行结果均由本人在真实运行的系统上采集与核对。

11.3 人工责任与边界

本人对最终提交内容负全部责任：理解并能够解释每一处核心代码的意图；亲自运行 typecheck、lint、test、build 与提交检查并确认结果；对 AI 给出的建议进行筛选、修改与整合，凡是不符合项目边界或无法解释的实现一律不予采用。AI 工具未替代项目构思、代码理解、调试责任与最终提交责任。

11.4 合规与隐私

- ☐ 不提交无法解释的 AI 生成核心代码；不隐瞒 AI 在代码、Prompt 或报告整理中的参与。
- ☐ 不编造功能、运行结果、公网链接、测试数据或来源；本报告中的截图与数据均来自真实运行的本地系统。

□ 不向外部服务提交包含密钥、个人隐私或未脱敏目标的内容；DeepSeek 密钥仅在本地服务端环境变量中读取。

需要特别说明的是：本人能够离开 AI 独立解释项目中的每一个关键决策——为什么把编排器设计成线性状态机、为什么所有失败路径都要降级而不是抛错、为什么时间要做确定性格式化、为什么主题状态要用 `useSyncExternalStore` 而不是普通的 `useEffect`。这些理解是在与 AI 协作、并亲手调试与重构的过程中真正内化的，而非简单复制粘贴。

综上，AI 工具在本项目中显著提升了开发与写作效率，但项目的构思、关键实现、调试验证与最终质量把关均由本人完成并负责。

附录 验证命令与目录结构

A.1 一键全量验收

```
pnpm typecheck && pnpm lint && pnpm test && pnpm check:submit && pnpm build
```

A.2 主要目录

- ☐ src/app/ Next.js 页面与 API 路由
- ☐ src/components/ 规划器、设置、导出与系统级（主题/命令面板）组件
- ☐ src/lib/agent/ Agent 编排、Prompt、记忆、JSON 解析
- ☐ src/lib/search/ SearXNG 客户端与可信度分级
- ☐ src/lib/export/ md/docx/pdf/xlsx/bundle 导出
- ☐ src/lib/db/ Drizzle schema、查询与初始化
- ☐ scripts/ 一键启动、原生模块自检、提交检查、报告与提交包生成
- ☐ assets/fonts/ PDF 中文字体（SimHei.ttf）
- ☐ docs/ 审计文档、报告与报告插图